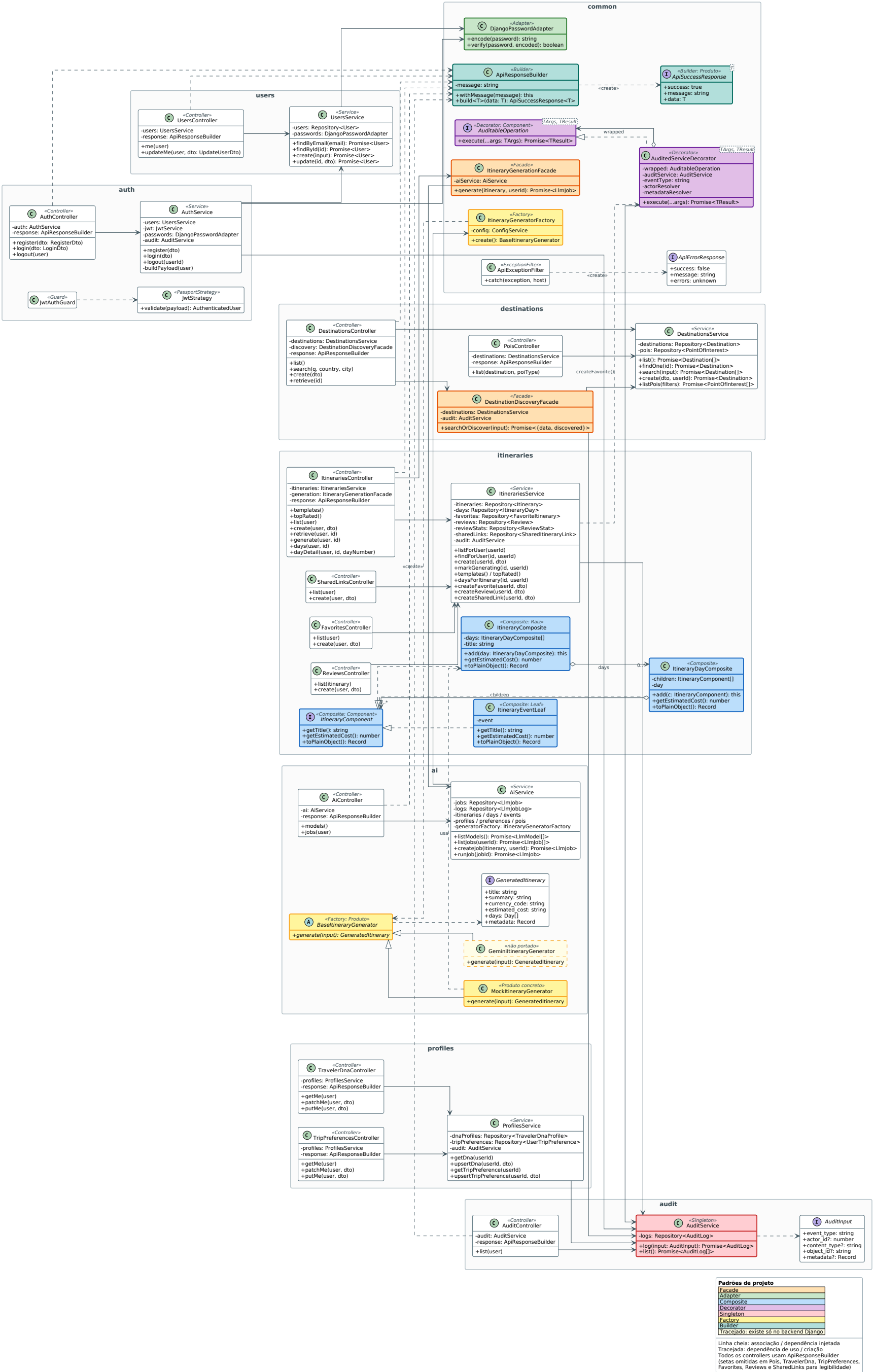
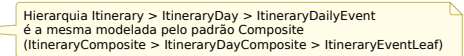


Diagrama de classes — visão geral

Todos os módulos do backend NestJS (controllers, services, interfaces) com os participantes de cada padrão de projeto destacados por cor.

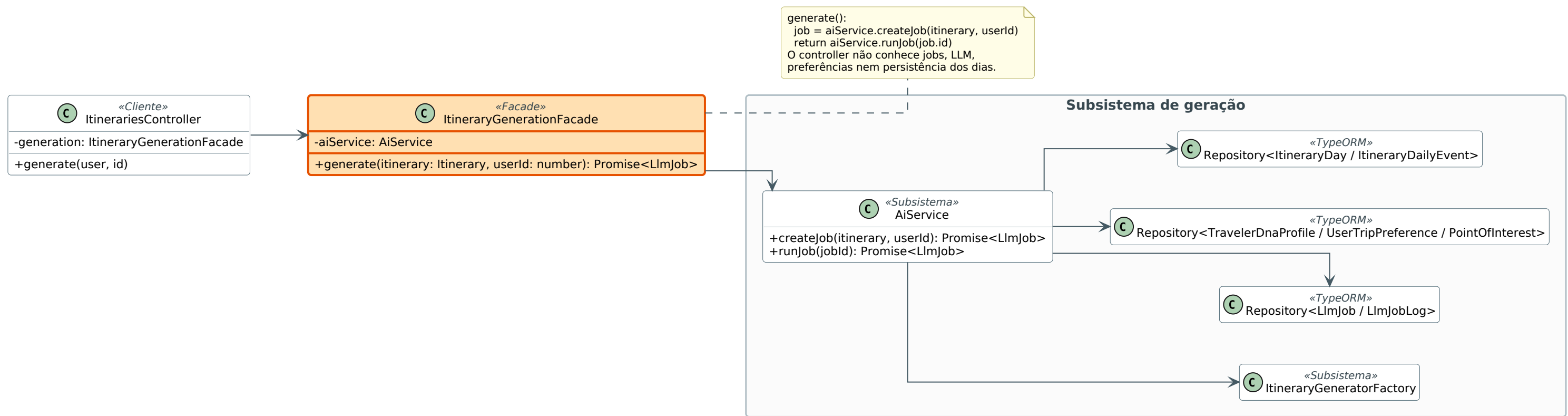


Entidades persistidas e seus relacionamentos, mapeadas sobre o schema PostgreSQL herdado do Django.



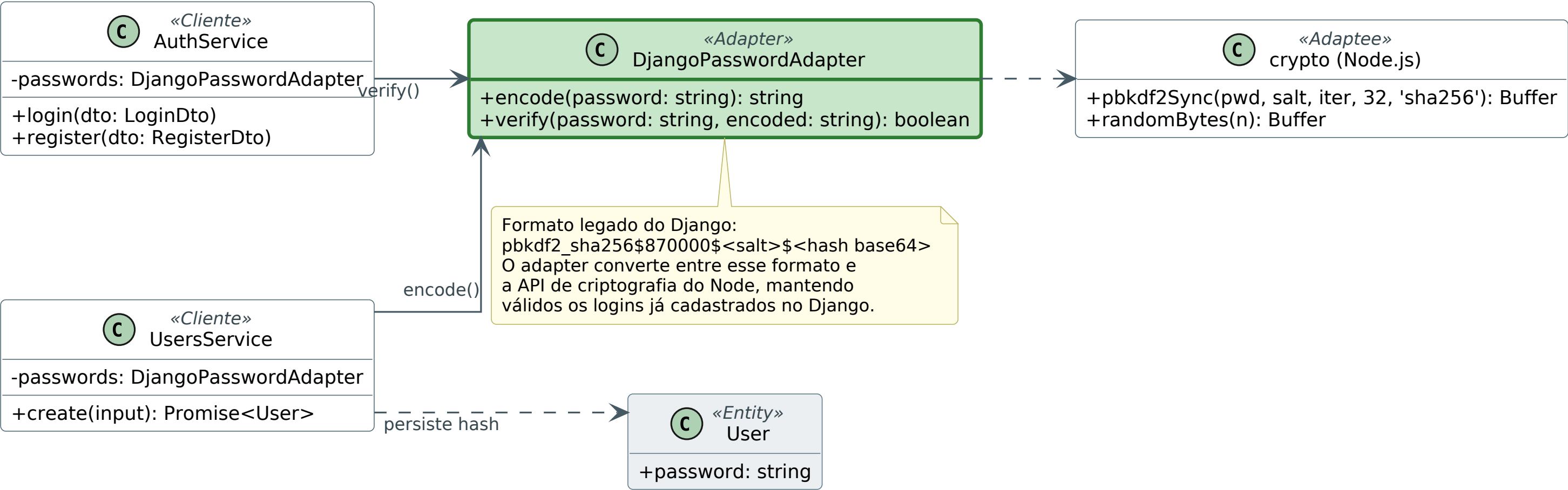
Padrão Facade — ItineraryGenerationFacade

O controller chama um único método generate(); a fachada coordena criação e execução do job de LLM no AiService.



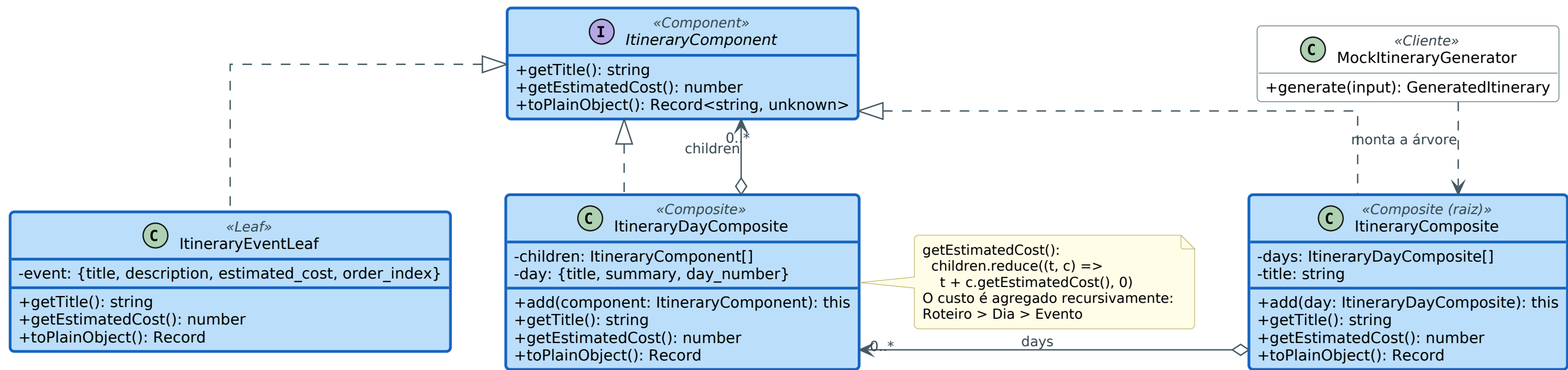
Padrão Adapter — DjangoPasswordAdapter

Traduz o formato de hash pbkdf2_sha256 do Django para a API de criptografia do Node, preservando os logins existentes.



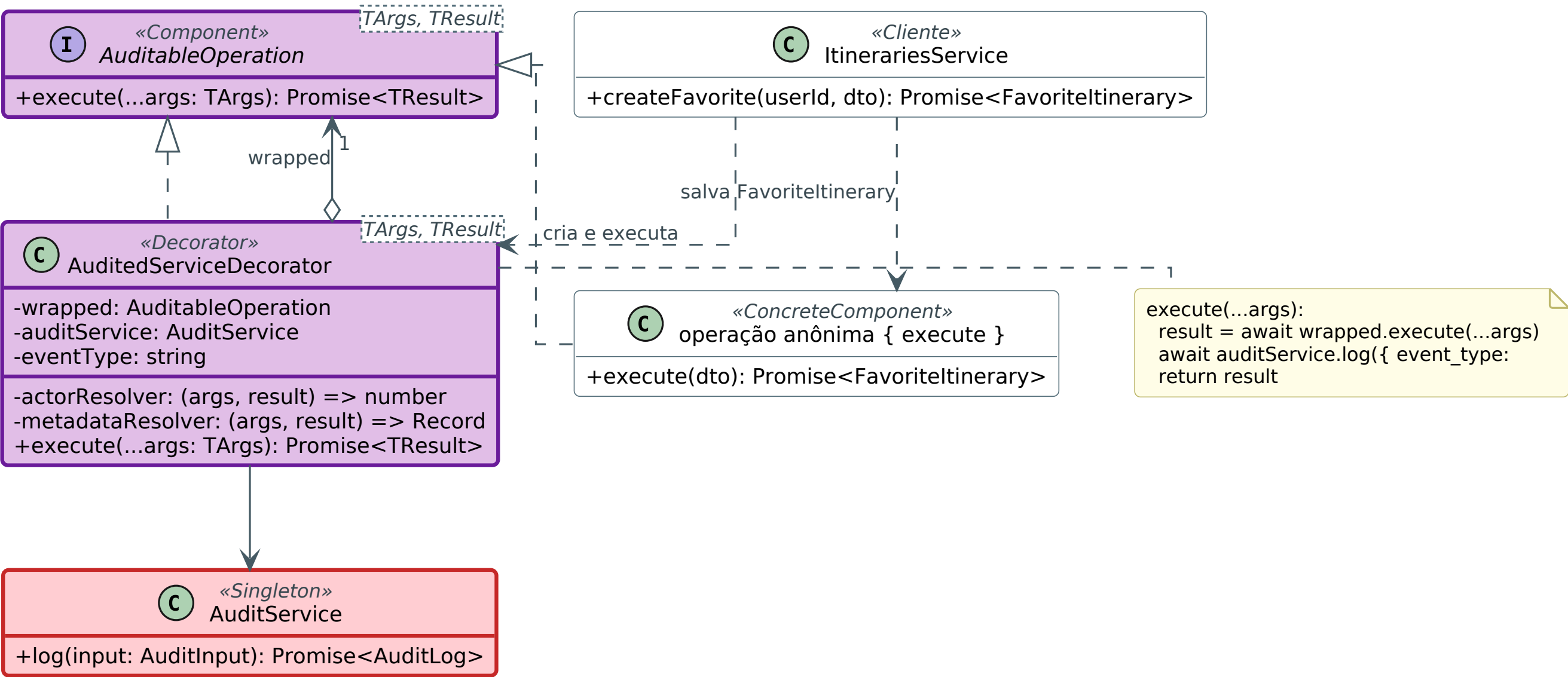
Padrão Composite — estrutura do roteiro

Roteiro, dia e evento compartilham a interface ItineraryComponent; o custo total é calculado recursivamente.



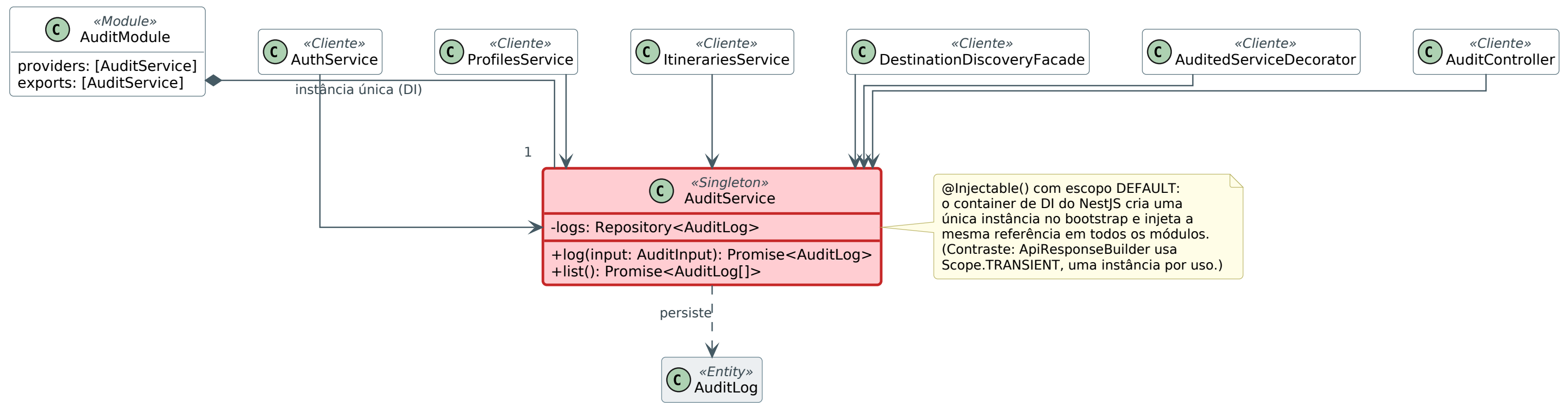
Padrão Decorator — AuditedServiceDecorator

Envolve uma operação (salvar favorito), executa a regra original e registra a auditoria, sem alterar a operação decorada.



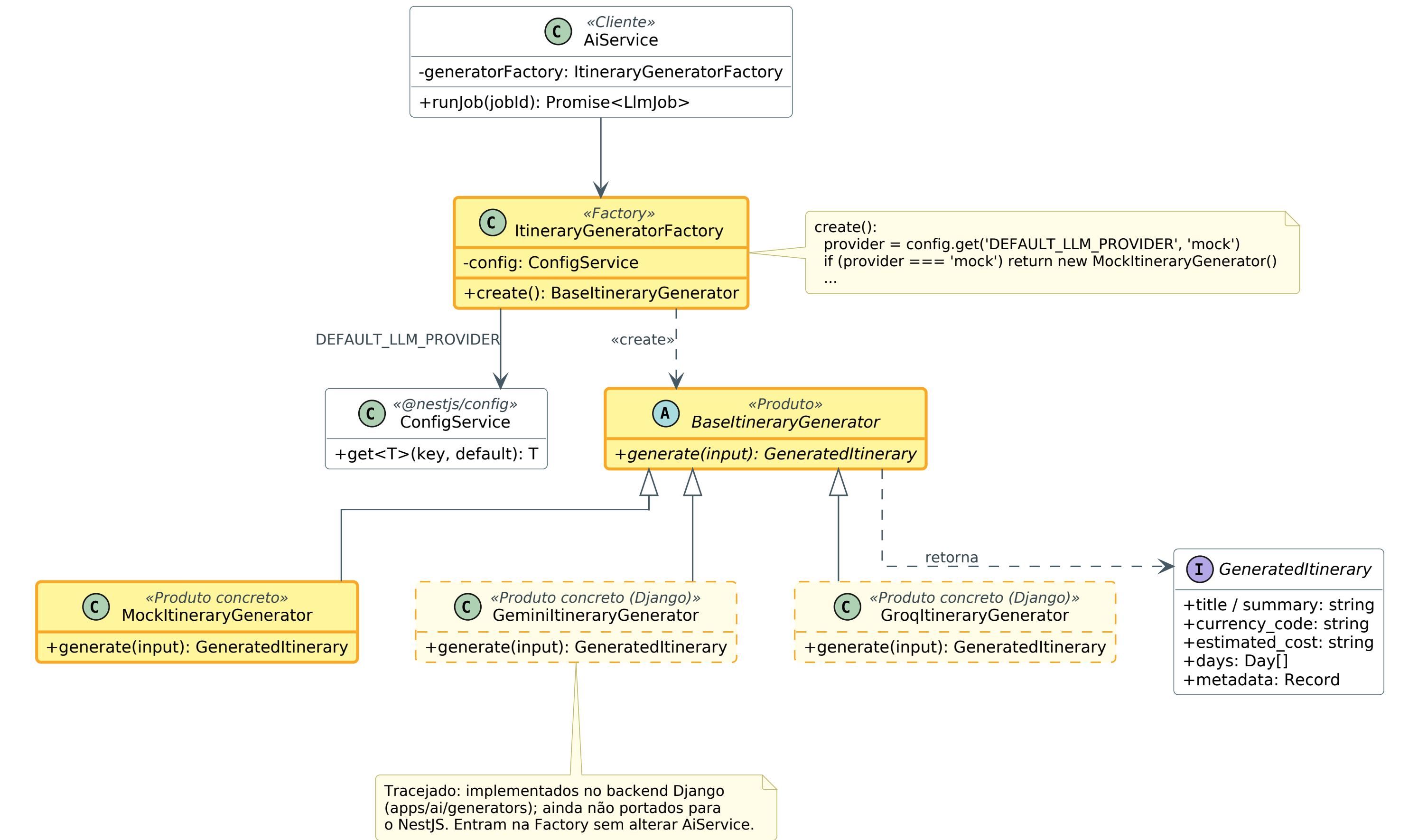
Padrão Singleton — AuditService

Uma única instância, criada e gerenciada pelo container de injeção de dependência do NestJS, compartilhada por todos os módulos.



Padrão Factory — ItineraryGeneratorFactory

Escolhe a implementação de BaseltineraryGenerator a partir de DEFAULT_LLM_PROVIDER; o AiService depende só da abstração.



Padrão Builder — ApiResponseBuilder

Monta passo a passo o envelope { success, message, data } usado por todos os controllers.

